

SeedDown System Documentation

Last updated: 2026-05-22

1. Purpose

SeedDown is an AI and IoT vertical farming platform for two operating levels:

- Beginner farms: small home or learning farms with guided setup, package-based sensors, simple alerts, and friendly AI explanations.
- Commercial farms: multi-zone vertical farms with device assignment, live monitoring, 3D digital twin views, control actions, disease support, and resource planning.

The system is designed around one complete loop:

1. A user registers a farm and scans a package QR code.
2. The backend registers the ESP32 package and returns a device token.
3. The ESP32 or Wokwi sketch uploads sensor readings to the backend.
4. The backend stores readings in Firebase Firestore.
5. Threshold rules and AI services analyze the readings.
6. The frontend shows live dashboards, alerts, 3D status, and planning tools.
7. Manual or generated commands are stored for ESP32 polling.
8. The ESP32 polls commands and updates actuators or Wokwi output LEDs.

2. High-Level Architecture

SeedDown has seven practical layers.

Layer	Responsibility
Device and sensor layer	ESP32 or Wokwi package reads temperature, humidity, light, pH, water, gas, EC, CO2, water flow, energy, or camera context depending on package level.
Connectivity and identity layer	WiFi/HTTPS, QR serial onboarding, device token authentication, heartbeat, and reassignment.
Backend and services layer	Node.js and Express routes for auth, farms, devices, sensors, alerts, AI, what-if, community, and consumption analysis.
Database layer	Firebase Firestore stores users, farms, devices, readings, commands, alerts, preferences, community records, and generated data.
AI and rules layer	Deterministic thresholds plus Groq/Gemini-backed analysis for thresholds, disease, what-if, chat, and resource insights.
Frontend layer	Vite web app with beginner and commercial dashboards, 3D canvases, control panels, disease analysis, and community.
Actuation layer	Wokwi/ESP32-supported command outputs such as water pump, fan, buzzer, pH warning, fertilizer alert, CO2 low, gas alert, and optional light output.

Runtime flow:

```
User / QR
-> device registration
-> ESP32/Wokwi sensor upload
-> backend ingestion
-> Firestore sensorReadings
-> threshold / AI analysis
-> frontend dashboards and alerts
-> deviceCommands
-> ESP32 command polling
-> actuator response
```

3. Authentication and Session Flow

3.1 Register

The register form sends account details to `POST /api/auth/register`.

The backend:

1. Validates name, email, password, and account mode.
2. Hashes the password.
3. Stores a user document in Firestore.
4. Returns a JWT and user summary.

The frontend stores:

- `token`: JWT for authenticated backend requests.
- `seeddown_user`: cached user summary.
- `seeddown_mode`: beginner or commercial.
- `user_farms`: cached farm list after farm fetch.

3.2 Login

Login sends email and password to `POST /api/auth/login`.

On success:

1. The token is saved in `localStorage`.
2. User summary is cached.
3. Mode is restored.
4. The app moves to the farm list instead of staying on splash/login.

3.3 Refresh behavior

On app load, the frontend checks `localStorage`.

If a token exists:

1. The app hydrates mode, cached user, and cached farms.
2. The user is sent to the farm list.
3. The app calls `/api/auth/me` and `/api/farms` in the background.

4. If the token is invalid or expired, the app clears session cache and returns to login.

This prevents users from needing to log in again after every page refresh while still respecting token expiry.

3.4 Logout

Logout clears:

- token
- seeddown_user
- seeddown_mode
- user_farms
- farm_profile

Then the user returns to the unauthenticated flow.

4. Farm and Device Identity

4.1 Farm identity

Each farm has an application-level farm ID. Beginner farms are normally single-field farms. Commercial farms contain a farm master and multiple zones.

Important rule:

- New users should not inherit `farm_001`.
- `farm_001` remains only as Wokwi or legacy compatibility.
- Normal farms use their own generated or backend-created `farmId`.

4.2 QR serials

SeedDown QR serials identify package type.

Serial pattern	Meaning
SD-BGN-STR-xxxxx	Beginner Starter
SD-BGN-STD-xxxxx	Beginner Standard
SD-BGN-PRO-xxxxx	Beginner Pro
SD-COM-ZON-xxxxx	Commercial Zone Node
SD-COM-FRM-xxxxx	Commercial Farm Master

The QR flow calls `POST /api/devices/register`.

The backend resolves:

- package level
- device type
- device ID
- farm ID or target assignment

- device token
- active status

4.3 Device token

Real devices authenticate sensor upload with:

```
x-device-token: <deviceToken>
```

The token links a reading to the registered device and its farm or zone assignment. For commercial farms, active assignment is the canonical routing rule. Firmware `zoneId` can still be used as a fallback for Wokwi compatibility, but an active reassignment should control which zone receives new readings.

4.4 Commercial replacement flow

Commercial device assignment supports replacement:

1. User selects farm master or a zone.
2. User scans QR or enters serial manually.
3. The new device becomes active for that target.
4. Old device is marked replaced or inactive.
5. Historical readings stay stored.
6. Latest dashboard data prefers the active device.

This makes the flow realistic when an ESP32 breaks and the operator buys another one.

5. Sensor Payloads and Conversion

5.1 Backend sensor upload

ESP32 or Wokwi sends readings to:

```
POST /api/sensors
```

The backend also keeps compatibility with:

```
POST /api/sensors/data
```

The reading can include `deviceId`, `farmId`, `fieldId`, `zoneId`, `packageLevel`, and sensor fields. When `x-device-token` is present, backend device lookup can fill missing identity fields.

5.2 Canonical sensor fields

Canonical field	Common aliases	Unit / meaning
temperature	temp	Celsius. DHT temperature for air around the farm or zone.
humidity	humid, hum	Percent relative humidity from DHT.
lightRaw	light, light_raw, lux	Raw light sensor/ADC reading or simulated light level.
soilRaw	soil_raw, soilMoistureRaw	Raw soil moisture or substrate moisture reading.

Canonical field	Common aliases	Unit / meaning
ph	pH	Calibrated pH value.
phRaw	-	Raw pH analog reading before calibration.
waterDistanceCm	water, waterLevel	Ultrasonic distance from sensor to water surface, in cm.
gasRaw	gas, gasValue	Raw MQ gas sensor value.
ec	nutrientEc	Electrical conductivity in mS/cm.
ecRaw	-	Raw EC/TDS analog reading before calibration.
co2Ppm	co2, co2_ppm	CO2 concentration in ppm.
waterFlowLpm	flow, water_flow_lpm	Water flow in liters per minute.
energyKwh	powerKwh, energyKwh	Estimated or measured energy use in kWh.

5.3 Frontend normalization

The frontend uses a shared reading helper to avoid `NaN`, fake zeroes, or random fallback jumps.

Process:

1. Convert candidate values with finite-number parsing.
2. Treat `NaN`, `"NaN"`, empty strings, `Infinity`, `null`, and `undefined` as missing.
3. Read aliases into canonical fields.
4. Keep only readings with at least one real finite sensor value.
5. Display missing metrics as `--`.
6. Cache the last good reading where the page supports live fallback.

5.4 Sensor conversion notes

Temperature

Temperature is usually a direct DHT reading in Celsius.

Example:

```
{ "temperature": 24.6 }
```

Threshold use:

- Too high can trigger cooling/fan recommendation.
- Too low can trigger cold stress warning.

Humidity

Humidity is a direct DHT relative humidity percentage.

Example:

```
{ "humidity": 63 }
```

Threshold use:

- High humidity increases fungal disease risk.
- Low humidity can stress leafy crops.

Light

Light can be raw ADC or simulated Wokwi value.

SeedDown treats `lightRaw` as a relative brightness indicator. The dashboard and threshold engine compare it against package thresholds. In simple Wokwi setups, lower raw values may represent insufficient light depending on the sketch mapping.

Soil moisture

`soilRaw` is a raw analog reading. The exact wet/dry direction depends on the sensor module and calibration. SeedDown uses thresholds to interpret it for the chosen package.

General approach:

```
raw soil reading
-> compare against calibrated wet/dry threshold
-> water status
-> alert or water command recommendation
```

Water reservoir distance

`waterDistanceCm` comes from an ultrasonic sensor.

Conversion used by UI charts:

$$\text{water level percent} = (\text{tank depth cm} - \text{distance cm}) / \text{tank depth cm} * 100$$

SeedDown uses 30 cm as the current chart assumption when converting distance into percentage. Real deployment should calibrate this to the actual reservoir depth.

pH

`ph` is the preferred calibrated value. `phRaw` is raw analog data from a pH board.

Real pH sensors require calibration using known buffer solutions. In SeedDown:

- Calibrated pH is displayed directly.
- Out-of-range pH can create `PH_WARNING`.
- Disease and AI advisor can explain pH risk, but do not automatically dose correction unless a command feature is explicitly used.

EC / nutrient strength

`ec` is electrical conductivity in mS/cm. `ecRaw` is the raw sensor value.

EC represents nutrient strength. Too low can indicate weak nutrient solution; too high can indicate salt stress.

Gas

`gasRaw` comes from MQ-style gas sensing or simulation. It is treated as a risk indicator rather than a precise gas concentration unless calibrated hardware is used.

High gas risk can map to `GAS_ALERT`.

CO2

`co2Ppm` is CO2 concentration in parts per million.

Low CO2 can reduce photosynthesis. SeedDown can map this to `CO2_LOW` for commercial monitoring.

Water flow

`waterFlowLpm` indicates irrigation flow rate. It helps commercial farms confirm pump activity and detect abnormal flow.

Energy

`energyKwh` can come from a current sensor/energy meter or be estimated from actuator states.

Consumption analysis uses live readings where available and benchmark rules where AI or measured energy is unavailable.

6. Backend Ingestion and Storage

6.1 Sensor ingestion

When a reading arrives:

1. Backend reads `x-device-token` if present.
2. Device service resolves device identity and assignment.
3. Sensor service normalizes aliases such as `temp` to `temperature`.
4. The reading is written to Firestore with timestamps and context.
5. Threshold service checks the reading against preferences/package defaults.
6. If needed, an alert and command are generated.

6.2 Firestore collections

SeedDown uses Firestore collections conceptually like this:

Collection	Purpose
<code>users</code>	User account profile and mode.
<code>farms</code>	Beginner or commercial farm definitions.
<code>devices</code>	Registered QR/device token records and active status.
<code>sensorReadings</code>	Historical sensor data with farm/device/zone context.
<code>deviceCommands</code>	Pending and executed actuator commands.
<code>alerts</code>	Warning and risk events.
<code>preferences</code>	Thresholds and control preferences.
<code>community records</code>	Visits, posts, barter listings, and interactions.

Actual document shape can vary by route, but the frontend expects farm/device/zone context to exist on readings whenever possible.

6.3 Latest and history APIs

The frontend reads live data using:

```
GET /api/sensors/latest?deviceId=...
GET /api/sensors/latest?farmId=...
GET /api/sensors/latest?zoneId=...
GET /api/sensors/history?farmId=...&limit=48
```

Rules:

- Latest pages should show the newest valid reading.
- History pages should filter invalid rows out of charts.
- Missing values display as --.
- Startup should not use random simulator readings unless manually enabled.

7. Alerts and Commands

7.1 Alert generation

Alert routes:

```
POST /api/alerts/predict-beginner
POST /api/alerts/predict-commercial
```

The alert engine compares readings to package-aware thresholds.

Example alert logic:

- Temperature high -> ventilation/fan suggestion.
- Water low or soil dry -> water action suggestion.
- pH out of range -> pH warning.
- Gas high -> gas alert.
- CO2 low -> CO2 warning.

7.2 Supported command set

Wokwi and ESP32 command names should stay inside the supported set:

Command	Meaning
WATER_ON	Run water pump or water LED output.
LIGHT_ON	Turn grow light output on where the package supports it.
FAN_ON	Turn ventilation fan output on.
BUZZER_ON	Activate buzzer/alarm output.
PH_WARNING	Signal pH correction warning.
FERT_ALERT	Signal fertilizer/nutrient alert.
CO2_LOW	Signal low CO2 condition.
GAS_ALERT	Signal gas risk condition.
NO_ACTION	Store decision with no actuator action.

Manual override calls:

```
POST /api/sensors/command
```

ESP32 polling calls:

```
GET /api/sensors/command?deviceId=...&format=text
```

The command result can be acknowledged with:

```
POST /api/sensors/command-result
```

8. Frontend Runtime

8.1 Application startup

The web app starts from `frontend/js/main.js`.

Startup responsibilities:

1. Load saved mode and session cache.
2. Restore token/user/farm cache if available.
3. Route unauthenticated users to splash/login/register.
4. Route authenticated users to the farm list.
5. Register page modules and navigation handlers.
6. Avoid starting random IoT simulation by default.

8.2 API base

Frontend API calls use a shared API base helper. In production, Vercel should set:

```
VITE_API_BASE=https://your-render-backend-url
```

This prevents hardcoded localhost calls after deployment.

8.3 Live reading stability

Live cards and detail pages should follow this rule:

```
new valid reading -> update UI and cache
missing or invalid reading -> keep last good reading or show --
backend unavailable -> show cached timestamp or waiting state
```

This prevents strange jumps, `NaN`, fake zeroes, or unrelated fallback data.

9. Beginner Feature Flow

9.1 Launch, login, and register

Unauthenticated users see the launch, login, and register pages. SeedDown AI widgets should not appear here.

After login:

- Beginner users land in the farm list.
- If no farm exists, the farm list shows an empty state.
- The user creates a farm through Add New Field.

9.2 Add New Field

Beginner setup collects:

- farm name
- package type
- plant/crop layout
- optional photo or scan-based plant recognition
- QR/device registration if available

The saved farm contains its own farm ID and device ID. It should not become `farm_001`.

9.3 Beginner dashboard

The beginner dashboard shows:

- farm overview
- live sensor strip
- AI/NPC advisor after login
- alerts
- plants and field status
- navigation to sensor details and feature pages

Package level controls which sensors are visible. Starter should not pretend to have Pro-only sensors.

9.4 Beginner live data

The sensor strip reads from the selected farm/device. It normalizes incoming readings and keeps the last good value when fetches fail.

Expected behavior:

- Temperature and humidity display numeric values only when valid.
- Missing values display `--`.
- The UI shows a last-updated time.
- It does not jump to random startup simulator values.

9.5 Beginner 3D canvas

The beginner 3D farm view visualizes the field layout. It is an educational and monitoring view, not the source of truth for sensor data.

Data source priority:

1. selected farm layout
2. saved plant positions
3. live/cached sensor readings
4. empty state if no data exists

10. Commercial Feature Flow

10.1 Commercial farm setup

Commercial setup creates:

- farm identity
- zones
- plant counts per zone
- farm master assignment
- zone node assignments
- package-level capabilities

The commercial farm uses zone-level data for user-facing capacity and operational cards. The 3D rack model can keep its own internal visual capacity.

10.2 Commercial QR assignment

Commercial assignment supports:

- scan/take QR photo
- upload QR image
- manual serial fallback
- farm-level assignment
- zone-level assignment
- replacement of old devices

If a Wokwi zone node token such as `sd_demo_commercial_zone_node_1` is assigned to a selected zone, new readings from that token should display under the assigned zone.

10.3 Commercial dashboard

The commercial dashboard contains:

- top farm identity card
- facility overview card
- live zone health
- commercial 3D digital twin
- right-side operations and AI chat
- zone/sensor/tank detail interactions

Facility Overview should show live status metrics. If readings are unavailable, it should use -- or the last valid cached value with a timestamp.

10.4 Commercial 3D digital twin

The 3D canvas shows:

- racks and crops
- zone sensor markers
- tank/output markers
- SeedDown AI mascot
- selected object context

Rack and crop geometry should not change when sensor visuals are improved. Sensor/output objects are visual markers for monitoring.

10.5 SeedDown AI mascot

SeedDown AI appears only after login on commercial pages.

Behavior:

1. Default position is near the commercial scene.
2. User clicks zone, sensor, tank, or output.
3. Mascot walks along the ground toward that object.
4. Speech bubble explains what it is looking at.
5. Ask Now sends a contextual prompt to the existing AI chat.
6. User can hide the mascot; normal AI chat remains available.

10.6 Commercial AI chat

Commercial chat sends more than a message. It should include:

- current farm info
- selected zone or 3D object
- active device assignment
- latest or cached sensor reading
- thresholds/preferences where available
- recent chat history

Replies should be short, framed, and readable. If no live data exists, AI should say it is waiting for data instead of guessing.

11. Control Center

The Control page handles threshold preferences and manual override.

11.1 Threshold preferences

Users can adjust thresholds for supported metrics. Preferences are saved with:

```
PUT /api/sensors/preferences
```

Preferences are read with:

```
GET /api/sensors/preferences?deviceId=...
```

11.2 Manual override

Manual override should include:

- scope: farm level or zone
- target zone when scope is zone
- resolved device/target ID
- command from supported command set
- duration
- reason

The payload still stays compatible with existing command API fields:

```
{
  "deviceId": "commercial-zone-node-1",
  "command": "WATER_ON",
  "durationSeconds": 10,
  "reason": "Manual irrigation test",
  "scope": "zone",
  "zoneId": "zone_A"
}
```

12. Disease Analysis

Disease Analysis is advisory, not automatic control.

Flow:

1. User chooses Take Photo or Upload Photo.
2. User optionally answers symptom questions.
3. Frontend sends image/symptom context to [POST /api/ai/disease-analysis](#).
4. Backend AI service analyzes disease risk and returns likely issue, confidence, and recommendations.
5. UI displays diagnosis and suggested next steps.

Important rule:

- Disease diagnosis should not automatically send [FAN_ON](#), [WATER_ON](#), or chemical dosing commands.
- Any future "apply recommendation" feature should be explicit and safety-confirmed.

13. Camera Feature

Camera can work in two modes:

- Browser camera capture using [getUserMedia](#).

- Upload fallback when permission is denied or camera is unavailable.

Commercial flow:

1. User selects farm or zone camera context.
2. User starts camera or uploads photo.
3. User captures a frame.
4. Snapshot is stored in frontend state or backend storage path where implemented.
5. Snapshot can provide context for disease analysis or visual monitoring.

ESP32-CAM streaming is possible later if a stream URL is provided, but browser camera mode is enough for current web demo and local testing.

14. What-If and Resource Planning

What-If helps compare decisions before the user changes the farm.

Key endpoints:

```
POST /api/whatif/newplant
POST /api/whatif/costsaving
POST /api/whatif/forecast
GET /api/whatif/market-prices
GET /api/whatif/recipes
```

What-If may use AI analysis. If AI is unavailable, it falls back to crop/resource benchmarks and should label those numbers as benchmark estimates.

Example flows:

- New plant: user selects a crop and quantity; SeedDown estimates resource impact and suitability.
- Cost saving: user compares energy/water changes.
- Market/resource: user checks crop price and planning signals.

15. ESG and Consumption Tracking

Consumption uses real farm readings, not forced demo farm IDs.

Flow:

1. Resolve selected farm or device.
2. Fetch sensor history with farmId or deviceId.
3. Normalize and filter readings.
4. If no real readings exist, show waiting state.
5. If readings exist, calculate water, energy, CO2, savings, and eco grade.
6. Ask backend [/api/consumption/analysis](#) for AI/resource comparison.
7. If AI fails, show benchmark fallback clearly.

Consumption should not require a random Firestore collection for a deployed user. The correct source is the selected farm's real readings.

16. Community Hub

Community features include:

- user farm identity
- neighbor visits
- SOS/community posts
- comments and likes
- barter listings
- reserve and complete barter flow
- reward/interaction records

Community is independent from live sensor ingestion, but it can use farm context to make posts and visits meaningful.

17. Wokwi Validation

Wokwi validates the IoT loop before physical hardware is assembled.

Package folders:

```
iot/wokwi/beginner_starter
iot/wokwi/beginner_standard
iot/wokwi/beginner_pro
iot/wokwi/commercial_zone
iot/wokwi/commercial_master
iot/wokwi/commercial_zone_node_1
iot/wokwi/commercial_zone_node_2
iot/wokwi/commercial_zone_node_3
```

Each Wokwi package can contain:

- `sketch.ino`
- `diagram.json`
- `platformio.ini`
- `wokwi.toml`
- `src/main.cpp`

Wokwi upload flow:

1. Sketch connects to WiFi.
2. Sketch sends `POST /api/sensors`.
3. Header includes `x-device-token` where configured.
4. Payload includes sensor values and context such as `zoneId`.
5. Backend stores reading.
6. Dashboard fetches latest/history.
7. Sketch polls `/api/sensors/command`.
8. Sketch updates LED/output indicators.

Legacy Wokwi sketches may still use `farm_001`; current real-device flow should use QR registration and device token.

18. Real Hardware Mapping

18.1 Beginner Starter

Typical components:

- ESP32
- DHT11/DHT22
- light sensor or LDR
- simple wiring and enclosure

Reads:

- temperature
- humidity
- light

18.2 Beginner Standard

Adds:

- soil moisture
- pH module
- gas module
- ultrasonic/reservoir level

Reads:

- Starter metrics
- soil moisture
- pH
- gas
- water level

18.3 Beginner Pro

Adds:

- EC/TDS
- CO2

Reads:

- Standard metrics
- EC
- CO2

18.4 Commercial Zone Node

Typical zone node:

- ESP32
- DHT sensor
- soil/substrate sensor
- light sensor
- pH and EC
- CO2 or gas sensor
- water flow sensor
- optional camera context

Reads zone-level crop conditions.

18.5 Commercial Farm Master

Typical farm master:

- ESP32
- reservoir sensor
- farm-level CO2 or gas
- energy/current sensor
- relay/MOSFET control outputs

Reads or controls farm-level infrastructure.

19. Deployment

19.1 Frontend

Frontend is a Vite app deployed to Vercel.

Production requirement:

```
VITE_API_BASE=https://your-render-backend-url
```

19.2 Backend

Backend is a Node.js/Express app deployed to Render.

Important environment variables:

```
JWT_SECRET=...  
FIREBASE_PROJECT_ID=...  
FIREBASE_SERVICE_ACCOUNT_JSON=...  
GROQ_API_KEY=...  
GEMINI_API_KEY=...  
GEMINI_API_KEY_2=...  
DA3_SERVICE_URL=...
```

AI keys can be omitted for fallback behavior, but Firebase and JWT settings are required for real backend operation.

20. Operational Rules

SeedDown should follow these rules in production:

1. Real user data should come from registered devices and selected farms.
2. Demo or legacy IDs should not override normal user farms.
3. Missing sensor values should show `--`, not `NaN`.
4. Empty farm history should show waiting states, not random readings.
5. AI output should be short, framed, and tied to current farm context.
6. Disease diagnosis should remain advisory.
7. Manual commands should stay inside supported Wokwi/ESP32 command names.
8. Device replacement should preserve historical readings.
9. Frontend should use `VITE_API_BASE` in production.
10. README should describe the product once; detailed technical flow belongs in this documentation.